CS 455 / CS 555 – Large Language Models

Spring 2025-2026

Sabancı University – Faculty of Engineering and Natural Sciences

Homework Assignment: Song2Vec

Building a Music Recommender System with Word Embeddings

Due Date:	Sunday, 29 March 2026, 23:59
Submission:	SUCourse+ (Individual Assignment)

1. Introduction

In this assignment, you will apply the concepts of **word embeddings** to build a music recommendation system. Instead of treating words in sentences as tokens, you will treat **songs in playlists** as your basic units. This approach, often called *"Song2Vec"* or *"Prod2Vec"* or *"Item2Vec"*, leverages the insight that songs appearing together in user-created playlists share semantic similarity, just as words appearing in similar contexts share meaning.

The core idea is elegant: if Word2Vec learns that "king" and "queen" are similar because they appear in similar textual contexts, then Song2Vec can learn that "Bohemian Rhapsody" and "Stairway to Heaven" are similar because they appear together in many classic rock playlists.

1.1 Learning Objectives

- Understand how Word2Vec can be generalized beyond NLP to recommendation systems
- Gain hands-on experience with Gensim's Word2Vec implementation
- Learn how hyperparameter choices differ between NLP and recommendation tasks
- Implement and evaluate a basic recommender system using learned embeddings
- Develop skills in failure analysis and systematic debugging of ML systems

1.2 Required Reading

Before starting this assignment, you **must** read the following paper:

> *Caselles-Dupré, H., Lesaint, F., & Royo-Letelier, J. (2018).* **"Word2Vec applied to Recommendation: Hyperparameters Matter."** *In Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18).* Available at: https://arxiv.org/abs/1804.04212

This paper demonstrates that optimal hyperparameters for recommendation tasks differ significantly from NLP defaults. You will explore these differences empirically in this assignment.

2. Dataset Description

You will use the **30Music Dataset**, a collection of listening sessions and playlists retrieved from Internet radio stations through the Last.fm API. This dataset was introduced by Turrin et al. (2015) and is widely used in music recommendation research.

2.1 Dataset Source and Access

- **Official Source:** https://remaplab.deib.polimi.it/resources/ (however, you can find necessary files in our Google Drive homework folder - **Data** folder)

2.2 Dataset Contents

The 30Music dataset contains several files. For this assignment, you will primarily work with:

File	Description
`playlist.idomaar`	User-created playlists with track IDs (PRIMARY)
`tracks.idomaar`	Track metadata: 'Artist/_/Title' in name field (REQUIRED)
`persons.idomaar`	Artist info (OPTIONAL - artist names are already in tracks file)

2.3 Data Format

The .idomaar files use a **tab-separated format** with JSON-encoded fields. Each line in `playlist.idomaar` represents one playlist with the following structure:

Column	Field	Description
1	type	Always "playlist" for this file
2	playlist_id	Unique identifier for the playlist
3	timestamp	Unix timestamp of playlist creation
4	playlist metadata	JSON with ID, Title, numtracks, duration
5	subjects/objects	JSON with user info and track IDs (integers)

Example playlist.idomaar line:

```
playlist\t0\t1216545588\t{"ID":2973549,"Title":"my_favorites",...}\t{"subjects":[{"type":"user
","id":41504}], "objects":[{"type":"track","id":3006631},
{"type":"track","id":1885124},...]}
```

Example tracks.idomaar line:

```
track\t174145\t-1\t{"name":"Andor+Foldes/_/Sonatine+%C3%BCber+Themen",...}\t{...}
```

> **Data Pipeline:** playlist.idomaar → track IDs → tracks.idomaar → 'Artist/_/Title' in name field. Use `urllib.parse.unquote_plus()` to decode URL encoding (+ = space, %XX = special chars), then split on '/_/' to get artist and title.

2.4 Dataset Statistics

Total Playlists	~57,000
Total Listening Events	~5.6 million
Unique Tracks	~600,000+
Unique Artists	~100,000+

3. Tasks and Requirements

Part A: Data Preparation and Exploration (20 points)

A.1 Data Loading and Preprocessing (10 points)

- Parse the playlist data and extract track sequences from each playlist
- Create a unique track identifier by combining artist and track name: `"artist_name__track_title"`
- Normalize track names: lowercase, handle special characters consistently
- Convert playlists to "sentences": each playlist becomes a list of track identifiers, *preserving the original order*
- Implement train/test split: 80% training, 20% testing (split at playlist level, not track level)

A.2 Exploratory Data Analysis (10 points)

Generate visualizations and statistics for the following:

- **Playlist length distribution:** Histogram showing number of tracks per playlist
- **Track frequency distribution:** Log-log plot showing the "long tail" of track popularity
- **Top-K analysis:** List top 20 most frequent tracks and top 20 most frequent artists
- **Vocabulary statistics:** Total unique tracks, tracks appearing only once (hapax legomena), coverage analysis

Part B: Song2Vec Model Training (25 points)

B.1 Baseline Model (10 points)

Train a baseline Word2Vec model using Gensim with the following default parameters:

```
model = Word2Vec(    sentences=playlists_as_sentences,    vector_size=100,
window=5,    min_count=5,    sg=1, negative=5,    ns_exponent=0.75, workers=4,
epochs=5)  # Important: more epochs = better embeddings
```

B.2 Hyperparameter Experiments (15 points)

Based on the findings of Caselles-Dupré et al. (2018), systematically explore the following hyperparameters. You **must** test at least the specified values:

Parameter	Values to Test	Why It Matters

`sg`	0 (CBOW), 1 (Skip-gram)	Architecture choice
`window`	5, 20, 40	Context range in playlist
`negative`	5, 10, 15	Number of negative samples
`ns_exponent`	-1.0, -0.5, 0, 0.5, 1.0	CRITICAL: Negative sampling distribution
`min_count`	3, 5, 10	Vocabulary pruning threshold
`vector_size`	50, 100, 300	Embedding dimensionality
`epochs`	5, 20, 50	Training iterations over corpus

> **Key Insight:** The `ns_exponent` parameter controls the negative sampling distribution. The NLP default (0.75) favors sampling frequent words as negatives. For recommendation, negative values (e.g., -0.5) sample *unpopular* items as negatives, which sometimes improves performance on long-tail items.

Part C: Evaluation (25 points)

C.1 Next-Song Prediction Task (15 points)

Implement **TWO** evaluation approaches and compare their results:

Approach 1: Context Averaging (Our Approach)

- For each playlist in the test set, use all N-1 songs as context
- Compute the average embedding of all context songs
- Retrieve top-K most similar songs to the average embedding
- Check if the actual last song appears in the top-K predictions

Approach 2: Single Query Song (Paper Approach)

Based on Caselles-Dupré et al. (2018), use only the **(n-1)th song** to predict the **nth song**:

- Use the second-to-last song as the query
- Find top-K songs most similar to the query song's embedding
- Check if the last song appears in the top-K predictions

> **Important:** You should observe that the Paper Approach yields higher HR@K and NDCG@K scores. In your analysis, explain *why* this is the case. Consider: information loss from averaging, task difficulty, and how each approach models user intent.

Required metrics for BOTH approaches:

- **Hit Rate @ K (HR@K):** Proportion of test cases where the true song appears in top-K
- **NDCG @ K:** Normalized Discounted Cumulative Gain accounting for rank position

Report these metrics for **K = 5, 10, 20, 50**

C.2 Qualitative Analysis (10 points)

- Select 5+ seed songs (include well-known tracks like "queen__bohemian rhapsody", "led zeppelin__stairway to heaven") and show their top-10 most similar songs using `model.wv.most_similar()`
- Demonstrate "song algebra" examples (analogous to king - man + woman = queen)
- Create a t-SNE or UMAP visualization of embeddings, colored by artist or genre (if available)

Part D: Recommender System (20 points)

D.1 Recommendation Interface (10 points)

Implement a function that takes a list of seed songs and returns recommendations:

```
def recommend_songs(seed_songs: List[str], model, top_k: int = 10) -> List[str]:

"""
Given a list of seed songs, return top_k recommended songs.
Args:
      seed_songs: List of track identifiers (e.g., ["queen__bohemian_rhapsody"])
      model: Trained Word2Vec model
      top_k: Number of recommendations to return

Returns: List of recommended track identifiers

"""

# Your implementation here
```

D.2 Cold-Start Analysis (10 points)

Analyze the cold-start problem in your system:

- What percentage of unique tracks in your dataset are excluded due to min_count?
- What percentage of test set songs are out-of-vocabulary (OOV)?
- Propose and implement at least one strategy to handle OOV songs (e.g., artist-level fallback)
- Compare HR@10 for frequent songs (>100 occurrences) vs. rare songs (<10 occurrences)

Part E: Analysis Report (10 points)

E.1 Failure Analysis (Required)

As specified in the course syllabus, you **must** provide a **"Bad Cases" table** with at least **5-10 failure cases**. Use the following template:

Input/Query	Model Output	Expected	Error Type	Root Cause
Seed song	Wrong recs	Good recs	e.g., Genre drift	Why it failed

Common error types to consider:

- **Popularity bias:** Model always recommends popular songs regardless of query
- **Genre drift:** Recommended songs are from wrong genre/style
- **Cold-start failure:** Query song not in vocabulary
- **Artist over-representation:** All recommendations from same artist
- **Temporal mismatch:** Old songs matched with modern songs inappropriately

> For at least **two failure cases**, propose a fix (e.g., change hyperparameters, add filtering, post-processing) and show before vs. after evidence.

E.2 Reproducibility Evidence (Required)

Your submission must include:

- **Random seed:** Document the seed used for all random operations
- **Configuration file or cell:** All hyperparameters in one place
- **Executed notebook:** Submit .ipynb with all outputs visible
- **Training logs:** Include training time and loss progression (if available)

4. Context-Specific Questions

> ⚠️ **Important:** These questions require analysis of *your specific execution results*. Generic answers that could apply to any run will receive reduced credit. Copy-pasting AI-generated summaries without evidence from your logs is considered a violation of academic integrity.

Answer the following questions in your report:

Q1. Hyperparameter Analysis: Compare performance for `ns_exponent = 0.75` (NLP default) vs. `ns_exponent = -0.5` (recommendation setting). Which performs better on your data? Report exact HR@10 values and explain the difference based on track frequency distribution in your dataset.

Q2. Playlist Length Effect: Split your test playlists into short (<10 songs), medium (10-30 songs), and long (>30 songs). Report HR@10 for each group. Which group shows the best performance and why?

Q3. Vocabulary Coverage: With `min_count=5`, how many unique tracks are in your vocabulary? What percentage of tracks appearing in your test set are out-of-vocabulary? How does changing min_count to 3 and 10 affect these numbers and HR@10?

Q4. Popular vs. Rare: For test playlists ending with a popular song (>100 occurrences in training) vs. rare song (<10 occurrences), what is the difference in HR@10? Does your best model improve performance on rare songs compared to the baseline?

Q5. Error Patterns: Looking at your failure cases, what is the most common error type? Propose a specific, implementable solution and explain why it would address this error pattern.

5. Submission Requirements and Deliverables

- **Jupyter Notebook (.ipynb):** Complete code with all cells executed and outputs visible
- **Analysis Report (.pdf):** Answers to context-specific questions, failure analysis table, visualizations
- **Prompt Log (.txt or .md or .pdf):** If AI tools were used, document all prompts and modifications (per course AI policy)
- **Trained Model (optional):** Best performing model saved with `model.save()`

6. Academic Integrity Reminder

As stated in the course syllabus, generative AI tools are **permitted** for this assignment under the **"Glass Box" policy**. However:

- You must document all AI usage in your Prompt Log
- The instructor reserves the right to conduct oral interviews about your code
- **Inability to explain your own code will result in a grade of zero**
- Plagiarism sanctions apply to all parties involved, as specified in the syllabus

> **Remember:** Running code successfully ≠ Understanding the code. Be prepared to explain every line.

Good luck!